

SKILL : WEEK 6

Task -1:

To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

CODE:

```
#include <stdio.h>
#include <string.h>

#define MAX 5
#define SIZE 50

int main()
{
    char history[MAX][SIZE];
    int count = 0;
    char command[SIZE];

    while (1)
    {
        printf("Enter command: ");
        fgets(command, SIZE, stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        if (count < MAX)
        {
            strcpy(history[count], command);
            count++;
        }
        else
        {
            for (int i = 1; i < MAX; i++)
                strcpy(history[i - 1], history[i]);

            strcpy(history[MAX - 1], command);
        }

        printf("\nHistory:\n");

        for (int i = 0; i < count; i++)
            printf("%d: %s\n", i + 1, history[i]);

        printf("\n");
    }

    if (count > 0)
        printf("Last command: %s\n", history[count - 1]);

    if (count <= MAX)
        printf("History is consistent.\n");

    return 0;
}
```

OUTPUT:

```
Radhakrishna@04:~# nano History.c
Radhakrishna@04:~# gcc History.c -o History
Radhakrishna@04:~# ./History
Enter command: ls

History:
1: ls

Enter command: pwd

History:
1: ls
2: pwd

Enter command: date

History:
1: ls
2: pwd
3: date

Enter command: whoami

History:
1: ls
2: pwd
3: date
4: whoami

Enter command: uname

History:
1: ls
2: pwd
3: date
4: whoami
5: uname

Enter command: exit
Last command: uname
History is consistent.
Radhakrishna@04:~# |
```

Task-2:

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    int pipefd[2];
    pid_t p1, p2;
    char *cmd1[] = {"ls", NULL};
    char *cmd2[] = {"wc", "-l", NULL};

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    p1 = fork();

    if (p1 == 0)
    {
        close(pipefd[0]);
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);

        execvp(cmd1[0], cmd1);
        perror("exec");
        return 1;
    }

    p2 = fork();

    if (p2 == 0)
    {
        close(pipefd[1]);
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);

        execvp(cmd2[0], cmd2);
        perror("exec");
        return 1;
    }

    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);

    printf("Pipeline executed successfully.\n");

    return 0;
}
```

OUTPUT:

```
Radhakrishna@04:~# nano pipeline.c
Radhakrishna@04:~# gcc pipeline.c -o pipeline
Radhakrishna@04:~# ./pipeline
46
Pipeline executed successfully.
Radhakrishna@04:~# |
```